

UNIT 4 : TUPLES, DICTIONARY, FILES

Tuples: Tuple Operations and methods, Dictionary: Dictionary Operations and methods; Files: Text Files, Reading and writing Files; Format Operator, Command line argument.

I. TUPLES

A tuple is a collection of objects which ordered and immutable. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating Python Tuples:

A tuple is created by placing all the items (elements) inside parentheses (), separated by commas. A tuple can have any number of items and they may be of different types (integer, float, list, string, etc.).

```
tup1 = ('physics', 'chemistry', 1997, 2000);
```

```
tup2 = (1, 2, 3, 4, 5);
```

```
tup3 = "a", "b", "c", "d";
```

What is Immutable in Tuples?

Tuples in Python are similar to Python lists but not entirely. Tuples are immutable and ordered and allow duplicate values. Some Characteristics of Tuples in Python.

- One cannot add items to a tuple once it is created.
- Tuples cannot be appended or extended.
- We cannot remove items from a tuple once it is created.

Accessing Values in Tuples:

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index.

For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);
```

```
tup2 = (1, 2, 3, 4, 5, 6, 7 );  
print "tup1[0]: ", tup1[0];  
print "tup2[1:5]: ", tup2[1:5];
```

Output:

```
tup1[0]: physics  
tup2[1:5]: [2, 3, 4, 5]
```

Example 2: var = ("Jerusalem", "College", "of Engineering")

```
print("Value in Var[0] = ", var[0])  
print("Value in Var[1] = ", var[1])  
print("Value in Var[2] = ", var[2])
```

Output:

```
Value in Var[0] = Jerusalem  
Value in Var[1] = College  
Value in Var[2] = of Engineering
```

Access Python Tuple Elements:

Like a list, each element of a tuple is represented by index numbers (0, 1, ...) where the first element is at index 0. We use the index number to access tuple elements.

1. Indexing

Indexing is used to access an item in a tuple, where the index starts from 0. So, a tuple having 6 elements will have indices from 0 to 5. The index must be an integer.

Example: # accessing tuple elements using indexing

```
letters = ("p", "r", "o", "g", "r", "a", "m")  
print(letters[0]) # prints "p"  
print(letters[5]) # prints "a"
```

2. Negative Indexing

Python allows negative indexing for its sequences. The index of -1 refers to the last item, -2 to the second last item and so on.

For example:

```
# accessing tuple elements using negative indexing
```

```
letters = ('p', 'r', 'o', 'g', 'r', 'a', 'm')
```

```
print(letters[-1]) # prints 'z'
```

```
print(letters[-3]) # prints 'm'
```

3. Slicing

We can access a range of items in a tuple by using the slicing operator colon

```
# accessing tuple elements using slicing
```

```
my_tuple = ('p', 'r', 'o', 'g', 'r', 'a', 'm')
```

returns a tuple with elements from index 1 to index 3.

```
# elements 2nd to 4th index
```

```
print(my_tuple[1:4]) # prints ('r', 'o', 'g')
```

returns a tuple with elements from beginning to index 2

```
# elements beginning to 2nd
```

```
print(my_tuple[:5]) # prints ('p', 'r')
```

returns a tuple with elements from index 5 to the end

```
# elements th to end
```

```
print(my_tuple[5:]) # prints ('a', 'm')
```

returns all tuple items.

```
# elements beginning to end
```

```
print(my_tuple[:]) # Prints ('p', 'r', 'o', 'g', 'r', 'a', 'm')
```

Iterating through a Tuple in Python:

use the for loop to iterate over the elements of a tuple.

For example,

```
languages = ('Python', 'Swift', 'C++')
```

```
# iterating through the tuple
```

```
for language in languages:
```

```
    print(language)
```

OUTPUT:

```
Python
```

```
Swift
```

```
C++
```

Check if an Item Exists in the Python Tuple:

We use the **in** keyword to check if an item exists in the tuple or not.

For example,

```
languages = ('Python', 'Swift', 'C++')
```

```
print('C' in languages)  # False
```

```
print('Python' in languages)  # True
```

Here,

'C' is not present in languages, 'C' in languages evaluates to False.

'Python' is present in languages, 'Python' in languages evaluates to True.

Delete Tuple Elements:

Removing individual tuple elements is not possible.

To explicitly remove an entire tuple, just use the del statement.

For example –.

```
tup = ('physics', 'chemistry', 1997, 2000)

print tup

del tup

print "After deleting tup : "

print tup
```

This program raises an Exception.

Basic Tuples Operations

Tuples respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new tuple, not a string. In fact, tuples respond to all of the general sequence operations

Python Expression	Results	Description
len((1, 2, 3))	3	Length
(1, 2, 3) + (4, 5, 6)	(1, 2, 3, 4, 5, 6)	Concatenation
('Hi!') * 4	('Hi!', 'Hi!', 'Hi!', 'Hi!')	Repetition
3 in (1, 2, 3)	True	Membership
for x in (1, 2, 3): print x,	1 2 3	Iteration

Code for concatenating 2 tuples

```
tuple1 = (0, 1, 2, 3)
```

```
tuple2 = ('Python', 'Program')
```

concatenating above two

```
print(tuple1 + tuple2)
```

Output:

```
(0, 1, 2, 3, 'Python', 'Program')
```

Nesting of Python Tuples:

A nested tuple in Python means a tuple inside another tuple.

```
# Code for creating nested tuples
```

```
tuple1 = (0, 1, 2, 3)
```

```
tuple2 = ('python', 'program')
```

```
tuple3 = (tuple1, tuple2)
```

```
print(tuple3)
```

Output : ((0, 1, 2, 3), ('python', 'program'))

Repetition Python Tuples:

We can create a tuple of multiple same elements from a single element in that tuple.

```
# Code to create a tuple with repetition
```

```
tuple3 = ('python',)*3
```

```
print(tuple3)
```

Output:

```
('python', 'python', 'python')
```

Finding the Length of a Python Tuple:

To find the length of a tuple, we can use Python's len() function and pass the tuple as the parameter.

```
# Code for printing the length of a tuple
```

```
tuple2 = ('python', 'program')
```

```
print(len(tuple2))
```

Converting a List to a Tuple:

We can convert a list in Python to a tuple by using the tuple() constructor and passing the list as its parameters.

Code for converting a list and a string into a tuple

```
list1 = [0, 1, 2]
print(tuple(list1))

# string 'python'
print(tuple('python'))
```

Output:

Tuples take a single parameter which may be a list, string, set, or even a dictionary (only keys are taken as elements), and converts them to a tuple.

(0, 1, 2)

('p', 'y', 't', 'h', 'o', 'n')

Built-in Tuple Functions:

Python includes the following tuple functions

Sr.No.	Function with Description
1	cmp(tuple1, tuple2) Compares elements of both tuples.
2	len(tuple) Gives the total length of the tuple.
3	max(tuple) Returns item from the tuple with max value.
4	min(tuple) Returns item from the tuple with min value.
5	tuple(seq) Converts a list into tuple.

Python Tuple Methods:

Python has two built-in methods that you can use on tuples.

Method	Description
<u>count()</u>	Returns the number of times a specified value occurs in a tuple
<u>index()</u>	Searches the tuple for a specified value and returns the position of where it was found

Advantages of Tuple over List in Python:

Tuples are quite similar to lists

Advantages of implementing a tuple over a list:

- Tuples for heterogeneous (different) data types and lists for homogeneous (similar) data types.
- Tuples are immutable, iterating through a tuple is faster than with a list. So there is a slight performance boost.
- Tuples that contain immutable elements can be used as a key for a dictionary. With lists, this is not possible.
- If you have data that doesn't change, implementing it as tuple will guarantee that it remains write-protected.

II. PYTHON DICTIONARY

Dictionaries are a useful data structure for storing data in Python because they are capable of imitating real-world data arrangements where a certain value exists for a given key.

The data is stored as key-value pairs using a Python dictionary.

- This data structure is mutable
- The components of dictionary were made using keys and values.
- Keys must only have one component.
- Values can be of any type, including integer, list, and tuple..

Example:

```
Dict = {1: 'Jerusalem ', 2: 'Engineering', 3: 'College'}  
print(Dict)
```

OUTPUT:

```
{1: 'Jerusalem ', 2: 'Engineering', 3: 'College'}
```

Accessing Values in Dictionary:

The keys of the dictionary can be used to obtain the values because they are unique from one another.

Example:

```
Employee = {"Name": "Dev", "Age": 20, "salary":45000,"Company":"WIPRO"}  
print(type(Employee))  
print("printing Employee data .... ")  
print("Name : %s" %Employee["Name"])  
print("Age : %d" %Employee["Age"])  
print("Salary : %d" %Employee["salary"])  
print("Company : %s" %Employee["Company"])
```

OUTPUT:

```
<class 'dict'>
printing Employee data ....
Name : Dev
Age : 20
Salary : 45000
Company : WIPRO
```

Adding Dictionary Values:

The dictionary is a mutable data type, and utilising the right keys allows you to change its values. Dict[key] = value and the value can both be modified. An existing value can also be updated using the update() method.

Example - 1:

```
# Creating an empty Dictionary
Dict = {}
print("Empty Dictionary: ")
print(Dict)

# Adding elements to dictionary one at a time
Dict[0] = 'Peter'
Dict[2] = 'Joseph'
Dict[3] = 'Ricky'
print("\nDictionary after adding 3 elements: ")
print(Dict)

# Adding set of values
# with a single Key
# The Emp_ages doesn't exist to dictionary
Dict['Emp_ages'] = 20, 33, 24
```

```
print("\nDictionary after adding 3 elements: ")
print(Dict)
# Updating existing Key's Value
Dict[3] = 'JavaTpoint'
print("\nUpdated key value: ")
print(Dict)
```

Output

Empty Dictionary:

```
{}
```

Dictionary after adding 3 elements:

```
{0: 'Peter', 2: 'Joseph', 3: 'Ricky'}
```

Dictionary after adding 3 elements:

```
{0: 'Peter', 2: 'Joseph', 3: 'Ricky', 'Emp_ages': (20, 33, 24)}
```

Updated key value:

```
{0: 'Peter', 2: 'Joseph', 3: 'JavaTpoint', 'Emp_ages': (20, 33, 24)}
```

Deleting Elements using del Keyword:

The items of the dictionary can be deleted by using the del keyword as given below.

Example:

```
Employee = {"Name": "David", "Age": 30, "salary":55000,"Company":"WIPRO"}
print(type(Employee))
print("printing Employee data .... ")
print(Employee)
print("Deleting some of the employee data")
del Employee["Name"]
del Employee["Company"]
print("printing the modified information ")
```

```
print(Employee)
print("Deleting the dictionary: Employee");
del Employee
print("Lets try to print it again ");
print(Employee)
```

Output:

```
<class 'dict'>
printing Employee data ....
{'Name': 'David', 'Age': 30, 'salary': 55000, 'Company': 'WIPRO'}
Deleting some of the employee data
printing the modified information
{'Age': 30, 'salary': 55000}
Deleting the dictionary: Employee
Lets try to print it again
NameError: name 'Employee' is not defined.
```

Note: The last print statement in the above code, it raised an error because we tried to print the Employee dictionary that already deleted.

Deleting Elements using pop() Method:

A dictionary is a group of key-value pairs in Python. You can retrieve, insert, and remove items using this unordered, mutable data type by using their keys.

Example:

```
# Creating a Dictionary
Dict1 = {1: 'JavaTpoint', 2: 'Educational', 3: 'Website'}
# Deleting a key
# using pop() method
pop_key = Dict1.pop(2)
```

```
print(Dict1)
```

Output:

```
{1: 'JavaTpoint', 3: 'Website'}
```

Additionally, Python offers built-in functions `popitem()` and `clear()` for removing dictionary items. In contrast to the `clear()` method, which removes all of the elements from the entire dictionary, `popitem()` removes any element from a dictionary.

Iterating Dictionary:

A dictionary can be iterated using for loop as given below.

Example 1:

```
# for loop to print all the keys of a dictionary
```

```
Employee = {"Name": "John", "Age": 29, "salary":25000,"Company":"WIPRO"}
```

```
for x in Employee:
```

```
    print(x)
```

Output

Name

Age

salary

Company

Example 2:

```
#for loop to print all the values of the dictionary
```

```
Employee = {"Name": "John", "Age": 29, "salary":25000,"Company":"WIPRO"} for x in Employee:
```

```
    print(Employee[x])
```

Output:

John

29

25000

WIPRO

Example 3:

#for loop to print the items of the dictionary by using items() method

```
Employee = {"Name": "John", "Age": 29, "salary":25000,"Company":"WIPRO"}
```

```
for x in Employee.items():
```

```
    print(x)
```

Output:

```
('Name', 'John')
```

```
('Age', 29)
```

```
('salary', 25000)
```

```
('Company', 'WIPRO')
```

Copy a Dictionary:

There are ways to make a copy, one way is to use the built-in Dictionary method copy()

Example:

```
thisdict = { "brand": "Ford", "model": "Mustang", "year": 1964}
```

```
mydict = thisdict.copy()
```

```
print(mydict)
```

Output:

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

Nested Dictionaries:

A dictionary can contain dictionaries, this is called nested dictionaries.

Example

```
myfamily = { "child1" : { "name" : "Email", "year" : 2004 },
```

```
    "child2" : { "name" : "Tobias", "year" : 2007 },
```

```
    "child3" : { "name" : "Linus", "year" : 2011 } }
```

Output:

```
{'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name': 'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year': 2011}}
```

Python Dictionary Methods:

Method	Description
<u>clear()</u>	Removes all the elements from the dictionary
<u>copy()</u>	Returns a copy of the dictionary
<u>fromkeys()</u>	Returns a dictionary with the specified keys and value
<u>get()</u>	Returns the value of the specified key
<u>items()</u>	Returns a list containing a tuple for each key value pair
<u>keys()</u>	Returns a list containing the dictionary's keys
<u>pop()</u>	Removes the element with the specified key
<u>popitem()</u>	Removes the last inserted key-value pair
<u>setdefault()</u>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<u>update()</u>	Updates the dictionary with the specified key-value pairs
<u>values()</u>	Returns a list of all the values in the dictionary

What are Dictionary Functions in Python?

Python provides several built-in functions for working with dictionaries.

Here are some dictionary functions in python :

- **len(dict):** Returns the length (i.e., the number of key-value pairs) of the dictionary.
- **dict[key]:** Returns the value associated with the specified key. If the key is not found, a `KeyError` is raised.
- **dict.get(key, default=None):** Returns the value associated with the specified key, or the default value if the key is not found. If the default parameter is not specified, it defaults to `None`.
- **dict.keys():** Returns a list of all the keys in the dictionary.
- **dict.values():** Returns a list of all the values in the dictionary.
- **dict.items():** Returns a list of all the key-value pairs in the dictionary, as tuples.
- **dict.clear():** Removes all key-value pairs from the dictionary.
- **dict.pop(key, default=None):** Removes the key-value pair with the specified key from the dictionary and returns the value. If the key is not found and the default is not specified, a `KeyError` is raised.
- **dict.popitem():** Removes and returns an arbitrary key-value pair from the dictionary. If the dictionary is empty, a `KeyError` is raised.
- **dict.update(other_dict):** Updates the dictionary with key-value pairs from another dictionary. If a key exists in both dictionaries, the value in the other dictionary overwrites the value in the original dictionary.
- **dict.fromkeys(seq, value=None):** Creates a new dictionary with keys from the specified sequence (e.g., a list, tuple, or set) and values set to the specified value. If the value parameter is not specified, it defaults to `None`.

Advantages of Dictionary Functions in Python

Python offers a wide range of built-in functions for working with dictionaries, which provide several advantages, including:

- **Quick access to dictionary keys, values, and items:** The built-in keys(), values(), and items() functions make it easy to access the keys, values, and key-value pairs in a dictionary, respectively. These functions allow you to quickly iterate over the contents of a dictionary and perform operations on the keys and values.
- **Efficient data manipulation:** Python's dictionary functions make it easy to manipulate data in a dictionary. For example, you can use the update() function to merge two dictionaries, the pop() function to remove a key-value pair from a dictionary, and the clear() function to remove all key-value pairs from a dictionary.
- **Efficient data lookup:** Python's dictionary functions allow you to efficiently look up data in a dictionary. For example, the get() function allows you to look up the value of a key in a dictionary, and the setdefault() function allows you to set a default value for a key if it doesn't already exist in the dictionary.
- **Convenient data conversion:** Python's dictionary functions make it easy to convert dictionaries to other data types, such as lists or tuples. For example, you can use the keys() and values() functions to convert a dictionary to a list of its keys or values, respectively.
- **Customizable sorting:** Python's sorted() function allows you to sort a dictionary by its keys or values, based on your desired criteria. This allows you to customize the sorting of data in a dictionary, which can be useful for certain applications.

Properties of Dictionary Keys

Dictionary values have no restrictions. They can be any arbitrary Python object, either standard objects or user-defined objects.

There are two important points to remember about dictionary keys –

- (a) More than one entry per key not allowed. Which means no duplicate key is allowed. When duplicate keys encountered during assignment, the last assignment wins
- (b) Keys must be immutable. Which means you can use strings, numbers or tuples as dictionary keys but something like ['key'] is not allowed.

III. FILES

FILES:

A file can be defined as a location for storing related data. A file holds a specific name for itself. A file can be a sequence of bits, bytes, lines or records depending on the application used to create it.

Example:

sample.py, student.txt – Here sample, student are the file names with file extensions

Types of files:

Files can be categorized as follows:

Text files:

A text file is a file containing characters structured as individual lines of text. In addition to printable characters, text files also contain the nonprinting newline character \n to denote the end of each text line

Binary files:

Binary files can contain various types of data, such as numerical values. These are not structured as lines of text. Such files can only be read and written via a computer program.

S.no	Type	Application
1.	Text	Notepad
2.	Audio	MediaPlayer
3.	Video	Mediaplayer
4.	Presentation	Mediaplayer
5.	Doc	Word

Different Modes to Open a File in Python

Mode	Description
R	Open a file for reading. (default)
W	Open a file for writing. Creates a new file if it does not exist or truncates the file if it exists.
X	Open a file for exclusive creation. If the file already exists, the operation fails.
A	Open a file for appending at the end of the file without truncating it. Creates a new file if it does not exist.
T	Open in text mode. (default)
B	Open in binary mode.
+	Open a file for updating (reading and writing)

FILE OPERATIONS:

Files in python can be manipulated via the following file operations:

1. Opening a file
2. Reading from a file
3. Writing to a file
4. Appending a file
5. Closing a file

Opening a file

All files must be opened first before they can be used. In python, when a file is opened, a file object is created that provides methods for accessing the file. In python there is a built in function `open()` to open a file.

Syntax:

```
file_object=open(filename,accessmode)
```

where file_object is used as a handle to the file

file_name-contains a string type value containing the name of the file which we want to access

access_mode-specifies the mode in which we want to open the file

```
file = open("File1.txt", "rb")  
print(file)
```

OUTPUT

```
<open file 'File1.txt', mode 'rb' at 0x02A850D0>
```

#Example for opening a file

```
fp1=open("sample.txt","w")  
print(fp1)  
fp=open("sample.txt","r")  
print(fp)
```

Output:

```
<_io.TextIOWrapper name='sample.txt' mode='w' encoding='cp1252'>  
<_io.TextIOWrapper name='sample.txt' mode='r' encoding='cp1252'>
```

#Example for opening a file when the file does not exist

```
fp1=open("sample1.txt","r")  
print(fp1)
```

Output:

```
fp1=open("sample1.txt","r")  
FileNotFoundError: [Errno 2] No such file or directory: 'sample1.txt'
```

PYTHON FILE OPENING MODES:

Modes	Description
R	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
Rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
W	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
Wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
A	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.

ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

Note: When opening a file if the access mode is not specified, the file is opened in read mode

File object attributes:

Once a file is opened and a file object is created, various pieces of information about the file can be gathered by using some predefined attributes. There are basically four types of attributes as shown below:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.
file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

```
file = open("File1.txt", "wb")
print("Name of the file: ", file.name)
print("File is closed.", file.closed)

print("File has been opened in ", file.mode, "mode")
```

OUTPUT

```
Name of the file: File1.txt
File is closed. False
File has been opened in wb mode
```

#File object attributes

```
fp=open("sample.txt","r")
print("Name of the file",fp.name)
print("closed or not",fp.closed)
print("opening mode",fp.mode)
print("File object",fp)
fp.close()
print("Name of the file",fp.name)
print("closed or not",fp.closed)
print("opening mode",fp.mode)
print("File object",fp)
```

Output:

```
Name of the file sample.txt closed ornot False opening mode r
File object <_io.TextIOWrapper name='sample.txt' mode='r' encoding='cp1252'>
Name of the file sample.txt closed ornot True
opening mode r
File object <_io.TextIOWrapper name='sample.txt' mode='r' encoding='cp1252'>
```

Reading from a file

In order to read from a file, the file must be opened in r mode. A file can be read in four different ways:

- read(size)
- readline()

- readlines()
- loop

read() method:

The read() enables reading the contents of an opened file.

Syntax:

```
fileobject.read()  
fileobject.read(size)
```

where size argument is optional. When the size is not specified, the entire file is read. When the size is specified, the size number of characters is read. When an empty file is read, no error is thrown but an empty string is returned.

readline() method:

This method is used for reading a file line by line. This method reads a file till the newline character including the newline character.

Syntax:

```
fileobject.readline() fileobject.readline(size)
```

where size is optional. When size is specified the size number of characters are read, otherwise it is similar to read()

readlines() method:

The readlines() method returns a list of lines of the entire file.

Syntax:

```
fileobject.readlines()  
fileobject.readlines(size)
```

When size is not specified, the entire file is read and each line is added as an element in a list.

Using loop:

A file can be read line by line using for loop

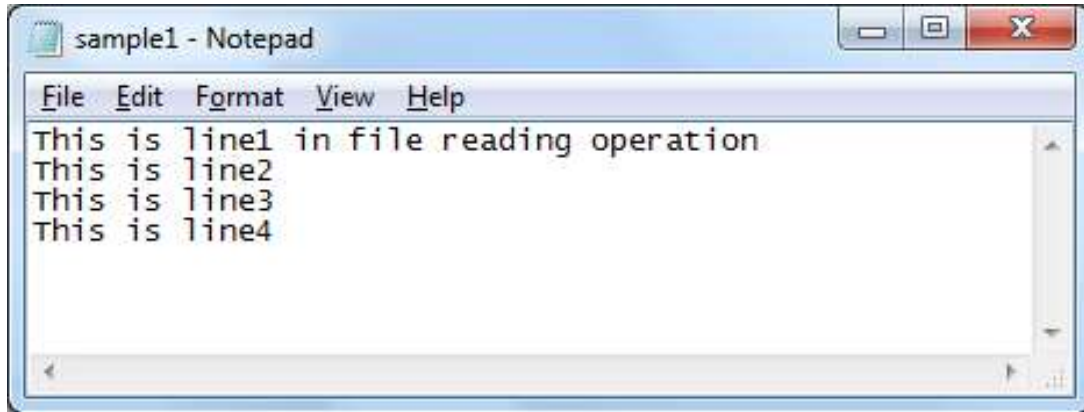
Syntax:

```
for line in <filevariable name>:
```

statements for file operations

Example for reading a file using various methods:

Let sample1.txt be the file

**#Reading a file using read() method**

```
fp=open("sample1.txt","r")  
print("Read a file using read()")  
print(fp.read())  
fp.close()
```

Output:

```
Read a file using read()  
This is line1 in file readingoperation  
This is line2  
This is line3  
This is line4
```

#Reading a file using read(size) method

```
fp=open("sample1.txt","r")  
print(fp.read(10))  
fp.close()
```

Output:

Read a file using read(size)

This is li

#Reading a file using readline() method

```
fp=open("sample1.txt","r")
print("Read a file using readline()")
print(fp.readline())
fp.close()
```

Output:

Read a file using readline()

This is line1 in file reading operation

#Reading a file using readlines()methods

```
fp=open("sample1.txt","r")
print("Read a file using readlines()")
print(fp.readlines())
fp.close()
```

Output:

Read a file using readlines()

['This is line1 in file reading operation\n', 'This is line2\n', 'This is line3\n', 'This isline4']

#Reading a file using loop `fp=open("sample1.txt","r")`

for line in fp:

```
    print("The contents of the file are")
print(line)
fp.close()
```

Output:

The contents of the file are
This is line1 in file reading operation
The contents of the file are
This is line2
The contents of the file are
This is line3
The contents of the file are
This is line4

Writing to a file

In order to write into a file, the file has to be opened in “w” mode or “a” mode or any other writing mode. The “w” mode overwrites the file if the file already exists. If the file does not exist, a new file is opened for writing.

Syntax:

```
fileobject=open(“filename”,“w”)
```

The write operation can be done using the following ways:

- write()
- writelines()

write() method:

The write method puts data into the file.
The write() method writes any string to an open file.
The write() method does not add a newline character (“\n”) to the end of the string

Syntax:

```
fileobject.write(string)
```

Where string is the content that is written to the file

writelines()

The writelines method puts multiple data into the file. The writelines() method writes any string to an open file.

Syntax : fileobject.writelines(string)

Example for writing a file when the file already exists

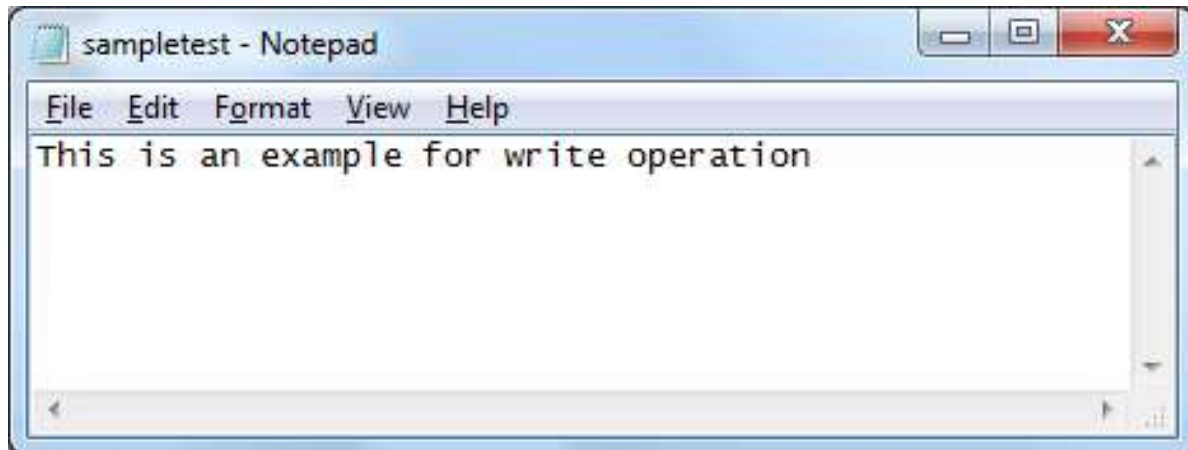
```
#Writing to a file when the file already exists fp=open("sampletest.txt","w")
```

```
fp.write("This content is written to the file")
```

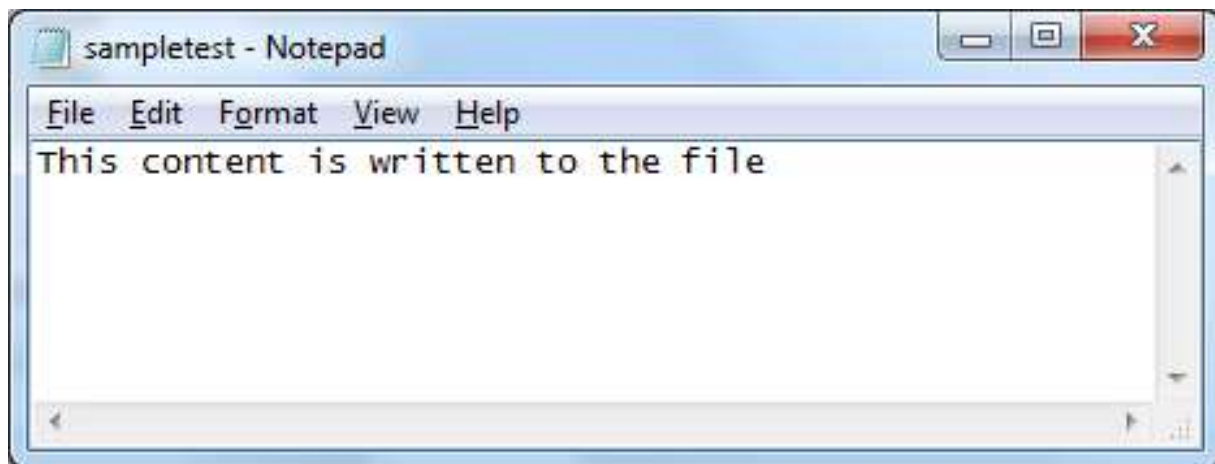
```
fp.close()
```

Output:

Before write operation



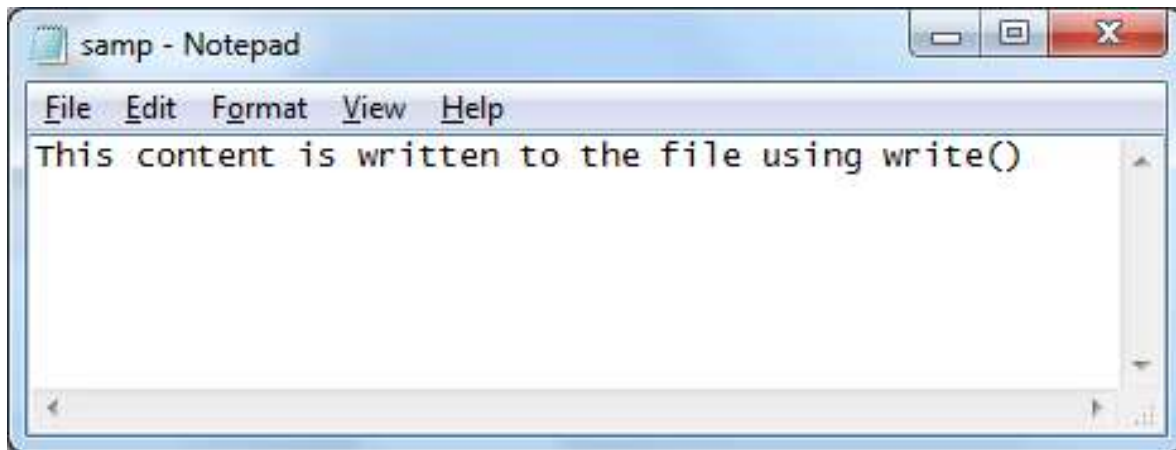
After write operation

**Example:****#Writing to a file using write()**

```
fp=open("samp.txt","w")
```

```
fp.write("This content is written to the file using write()")
```

```
fp.close()
```



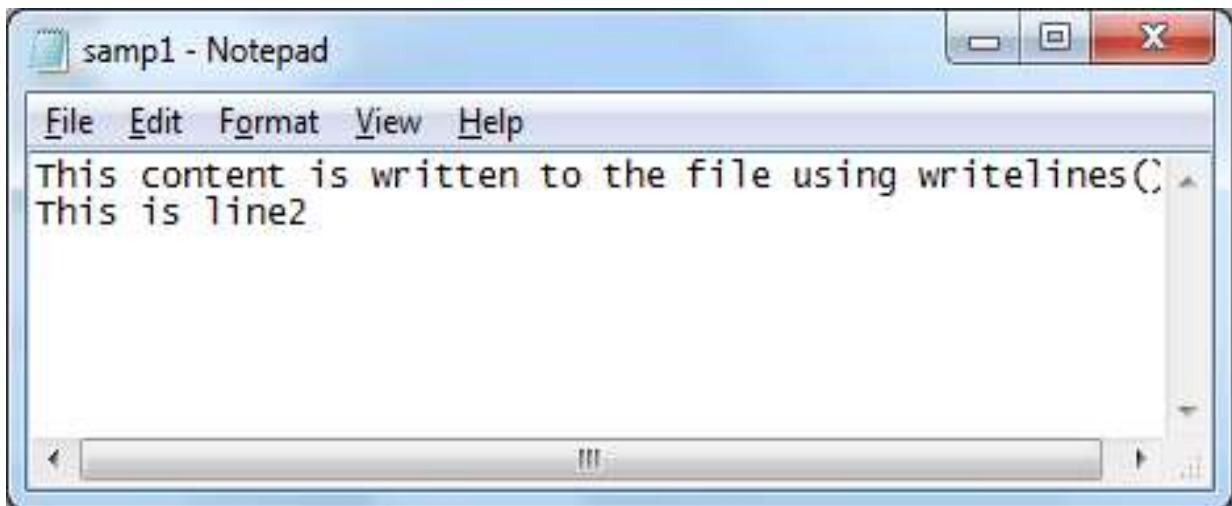
Example:

#Writing to a file using writelines()

```
fp=open("samp1.txt","w")
```

```
fp.writelines("This content is written to the file using writelines()\nThis is line2")
```

```
fp.close()
```

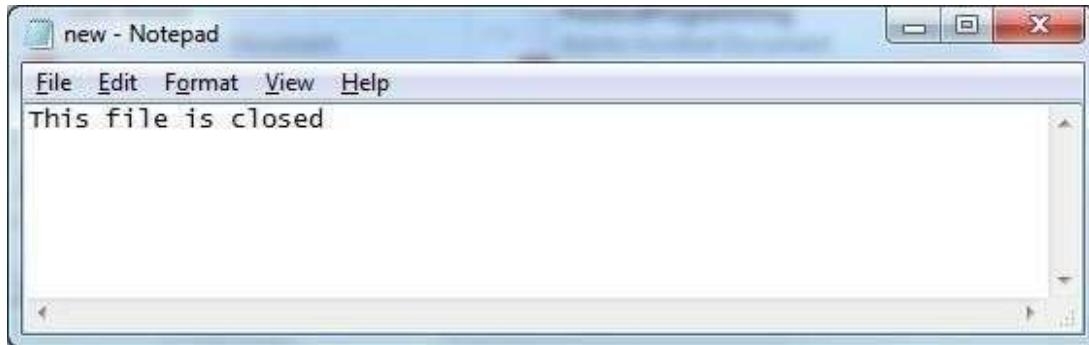


Closing a file:

When the operations that are to be performed on an opened file are finished, the file has to be closed in order to release the resources. Proper closing of file frees up the resources held with the file. The closing of file is done with the built-in function `close()`

Syntax:

```
fileobject.close()
```

Example:**#Closing a file**

```
fp=open("new.txt","r")  
print(fp.read())  
fp.close()
```

Output:

This file is closed

Closing a file using with:

A file can also be closed using the with statement as follows:

Syntax:

```
with open("filename",accessmode) as fileobject:statements for file operations
```

#Closing a file using with

```
with open("new.txt","r") as fp:print(fp.read())  
fp.close()
```

Output:

This file is closed

IV. FORMAT OPERATOR

The argument of write has to be a string, so if we want to put other values in a file, we have to convert them to strings. The easiest way to do that is with str()

Example:

```
>>> f5=open('stringsample.txt','w')
>>> f5.write(5)
TypeError: expected a string or other character buffer object
>>> f5.write(str(5))
```

An alternative is to use the **format operator**, %. When applied to integers, % is the modulus operator. But when the first operand is a string, % is the format operator.

The first operand is the **format string**, which contains one or more **format sequences**, which specify how the second operand is formatted. The result is a string.

For example, the format sequence '%d' means that decimal value is converted to string.

```
>>> run=8
>>> '%d'%run
```

The result is the string '8', which is not to be confused with the integer value 8

Format sequences:

Format Symbol	Conversion
%c	Character
%s	string conversion via str() prior to formatting
%i	signed decimal integer

%d	signed decimal integer
%u	unsigned decimal integer
%o	octal integer
%x	hexadecimal integer (lowercase letters)
%X	hexadecimal integer (UPPERcase letters)
%e	exponential notation (with lowercase 'e')
%E	exponential notation (with UPPERcase 'E')
%f	floating point real number
%g	the shorter of %f and %e
%G	the shorter of %f and %E

EXAMPLE:

```
#Example1
a="python"
print("%s"%a)
```

#Example2

```
runs=100
matches=3
years=2
print("arun scored %d in runs %d matches in %d years"%(runs,matches,years))
```

```
#Example3print("%s%r"%("hi","bye"))
```

```
#Example4print("%30s" %("I have space before"))
```

```
#Example5
```

```
print("%-30s"%("I have space after"))
```

```
    #formatting numbers
```

```
#Formattingnumbers
```

```
m=123.4548463749247247828n="number"
```

```
print("%s is %d"%(n,m))
```

```
print("%s is %f"%(n,m))
```

```
print("Truncate 5 characters %.5s"%m)
```

```
print("%s is %10d"%(n,m))
```

```
print("%s is %e"%(n,m))
```

```
#Format function
```

```
print("{1} {0}".format("Python","program"))
```

Output:

```
python
```

```
arun scored 100 in runs 3 matches in2 years hi'bye'
```

```
I have space before I
```

```
have spaceafter
```

```
number is 123
```

```
number is 123.454846
```

```
Truncate 5 characters 123.4
```

```
number is    123
```

```
number is 1.234548e+02
```

```
programPython
```

Float Precision With The Placeholder Method

- Floating-point numbers use the format **%a.bf**. Here,
- **a** would be the minimum number of digits to be present in the string;
- **bf** represents how many digits are to be displayed after the decimal point

Exampleprice = 70.56789

```
print('Price value = {}'.format(price))
```

```
print('Price value = %f'%price)
```

```
print('Price value = %.0f'%price)
```

```
print('Price value = %.2f'%price)
```

Output:

Price value = 70.56789

Price value = 70.567890

Price value = 71

Price value = 70.57

Python String format() Method

- The format() method formats the specified value(s) and insert them inside the string's placeholder. The format() method returns the formatted string.

Syntax

string.format(value1, value2...)

- The placeholder is defined using curly brackets: {}.
- The placeholders can be identified using named indexes {price}, numbered indexes {0}, or even empty placeholders {}.

Example:

#named indexes:

```
txt1 = "My name is {fname}, I'm {age}".format(fname = "John", age = 36)
```

#numbered indexes:

```
txt2 = "My name is {0}, I'm {1}".format("John",36)
```

#empty placeholders:

```
txt3 = "My name is {}, I'm {}".format("John",36)
```

```
print(txt1)
```

```
print(txt2)
```

```
print(txt3)
```

output

```
My name is John, I'm 36
```

```
My name is John, I'm 36
```

```
My name is John, I'm 36
```

V. COMMAND-LINE ARGUMENTS

It is possible to pass some values from the command line to your python programs when they are executed. These values are called command line arguments and many times they are important for your program especially when you want to control your program from outside instead of hard coding those values inside the code.

The command line arguments are handled using **sys** module. We can access command-line arguments via the **sys.argv**. This serves two purposes –

sys.argv is the list of command-line arguments. **len(sys.argv)** is the number of command-line arguments. Here **sys.argv[0]** is the program name ie. script name.

Example1:

```
import sys
print("There are %d arguments"%len(sys.argv))
print("Argument are",str(sys.argv))
print("File Name is:",sys.argv[0])
```

Output:

```
C:\Users\Dhivya>python cmdline.py
There are 1 argumentsArgument are ['cmdline.py']
File Name is: cmdline.py
```

```
C:\Users\Dhivya>python cmdline.py one two three
There are 4 arguments
Argument are['cmdline.py', 'one', 'two', 'three']
File Name is: cmdline.py
```

The first argument is always script name and it is also being counted in number of arguments. Here one, two, three are extra inputs passed to program through command line argument method while running python program `command_line.py`.

Example2:

```
Importsysc=sys.argvi=0
for a in c:
    print("Argument",i,"=",a)
    i=i+1
```

Output:

C:\Users\Dhivya>python cmdline1.py onetwo three

Argument 0 = cmdline1.py

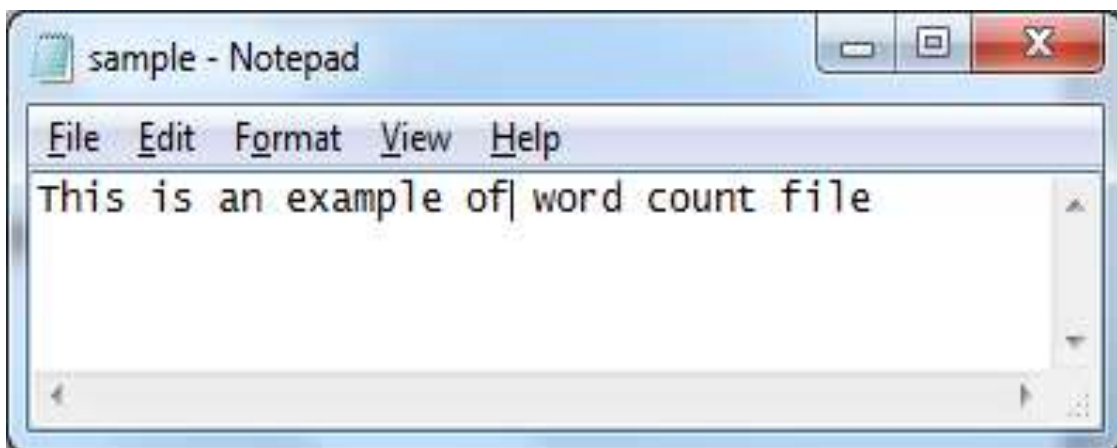
Argument 1 = one

Argument 2 = two

Argument 3 = three

Example3:**#Program using command line arguments for word count**

```
import sys
if len(sys.argv) != 2:
    print('Usage:./wc.py<filename>')sys.exit(1)
num_words = num_lines = num_chars = 0
with open(sys.argv[1]) as infile:
    for line in infile:
        num_lines += 1
        num_chars += len(line)
        line = line.strip()
        words = line.split()
num_words += len(words)
print('Number of Lines is %d' % num_lines)
print('Number of Words is %d' % num_words)
print('Number of Characters is %d' % num_chars)
```

OUTPUT:

C:\>python wordcount.py sample.txt Number of Lines is 1Number of Words is 8
Number of Characters is 37

Real-World Examples

Python scripts that use command line arguments can be found in many domains, from scientific computing to web development. Here are a few examples:

1. A script that converts a CSV file to JSON format: This script might take a CSV file as input, along with optional arguments specifying the output file format, the delimiter used in the CSV file, and the data types of the columns.
2. A script that downloads files from a remote server: This script might take a URL as input, along with optional arguments specifying the output directory, the number of concurrent downloads to perform, and the types of files to download.
3. A script that performs machine learning on a dataset: This script might take a dataset file as input, along with optional arguments specifying the type of machine learning algorithm to use, the hyperparameters to optimize, and the output file format.